Design and Implementation of a Prototype based in 2D Blocks with Multiple Shapes for Teaching Computer Programming to Visually Impaired Children

Lennin Paul Quiroz Villalobos
Department of Systems Engineering
Universidad de Lima
Lima, Peru
lquirozv@ulima.edu.pe

2021 IEEE Sciences and Humanities International Research Conference (SHIRCON) | 978-1-6654-2914-6/21/$31.00 ©2021 IEEE | DOI: 10.1109/SHIRCON53068.2021.9652298

***Abstract*— This paper proposes the implementation of a prototype based in physical 2D blocks with multiple shapes for teaching computer programming to visually impaired children. The image processing techniques used, the source code generation and compilation process, and design of human–machine interfaces are presented. The results showed that using space color change technique is possible to get a standard C++ code program since a development environment based in physical blocks where the compilation results can be communicated throws of voice. Finally, some examples of programs that can be implemented through the prototype and the corresponding conclusions are provided.**

Keywords— Visual Impairment, Computer Vision, Image Processing, Arduino, Innovation Education, TICs Education.

I. INTRODUCTION

Computational or algorithmic thinking helps us to be more proactive, more efficient, and less dependent. Moreover, computational thinking (CT) does not apply only to computers. For example, properly developed CT can help us make orderly, sequential, logical, and unambiguous decisions in all aspects of our lives. Therefore, digital literacy is now deemed a critical skill that must be taught from the early stages of human development, just like reading, writing, and mathematics (Gurises Unidos, 2011; Zapata-Ros, 2015).

In fact, the most efficient way to develop CT is through programming languages. For instance, the Code.org (https://code.org/) organization fosters and facilitates resources so that children at different school levels may have fun while learning programming. Furthermore, several programming environments, such as Alice, Greenfoot, and Scratch, are used to immerse beginners in the field of programming (Basogain Olabe, Olabe Basogain, and Olabe Basogain, 2015).

However, few resources exist for helping visually impaired children develop their CT through programming. One of these is Code Jumper, (https://codejumper.com/), a physical programming tool originally developed by Microsoft comprising colored blocks with multiple shapes, knobs, and buttons that are recognizable by touch. When joined through wires, these blocks constitute program functions that reproduce sounds upon execution (Linn, 2020).

Another example is the Osmo Coding platform (https://www.playosmo.com/en/coding/), developed by a TIDAL Lab team at Northwestern University. In this game, children help the main character, Awbie, collect as many strawberries as possible using physical blocks to provide instructions that the computer interprets.

Scottie Go! (https://scottiego.vicensvives.com/) is an educational game that helps children learn coding. It includes a series of cardboard tiles that players use to create programming commands and an application that will set the tasks, scan the solution, and tell Scottie and other characters where to go. (Hernández, 2019).

Melodic (https://epl.di.uminho.pt/~gepl/Melodic/), use a set of tactile blocks with different shapes, textures and meanings, the user controls the composition of the blocks as algorithms to form a musical sequence that produces the desired sound after the QR code each block is scanned by the mobile device. (Rui Costa, Cristiana Arújo, and Pedro Henriques, 2021).

Blocks4All (https://github.com/milnel2/blocks4alliOS) is a touchscreen application developed since the evaluation of five main accessibility problems in existing blocks based programming environments in touchscreen applications for children with visual impairments. From their findings authors also provide a design guideline for designing blocks-based environments and touchscreen applications for people with visual impairments (Lauren Milne, Richard Ladner, 2018)

Finally, the largest roadblock for helping visually impaired children develop CT is that their inability to use common tools as other children. Likewise, the few tools aimed at developing CT in children with visually impaired do it indirectly, through music, stories, or they are mobile applications, thereby losing the component of tactile learning of objets.

Within this context, this project proposes an alternative method—using image processing—for capturing a sequence of instructions provided by a child through physical blocks of different colors and shapes. Next, the program will be compiled and results reported in voice format.

II. METHODOLOGY

A. Proposed Architecture Diagram

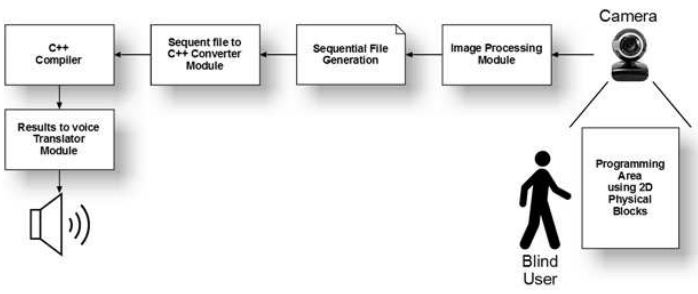


Figure 1: Proposed architecture diagram
Source: Prepared by the Author

978-1-6654-2914-6/21/$31.00 ©2021 IEEE

Authorized licensed use limited to: University of New South Wales. Downloaded on December 11,2025 at 01:41:18 UTC from IEEE Xplore. Restrictions apply.

Figure 1 denotes the general diagram of the proposed prototype implementation. The process starts in the programming area (canvas), where the user (a visually impaired child) interacts with physical programming blocks. Using these blocks, the child builds logical structures (programs) in the form of puzzles, differentiating block functions by their physical shapes and textures.

Then, during the program compilation stage, the programming area is scanned by a video camera that detects the blocks used and their logical order. Next, using image-processing techniques, the information scanned by the camera is decoded and translated into a sequential file that represents the captured information.

Afterwards, the sequential file is read by an interpreter program that converts this information into source code that will be automatically compiled and executed.

Finally, the compiler module executes the code generated by the interpreter module. The execution results are then converted into voice signals and reported orally to the user.

B. *Program Processing Stages*

Figure 2 depicts all the different stages required to process a program.

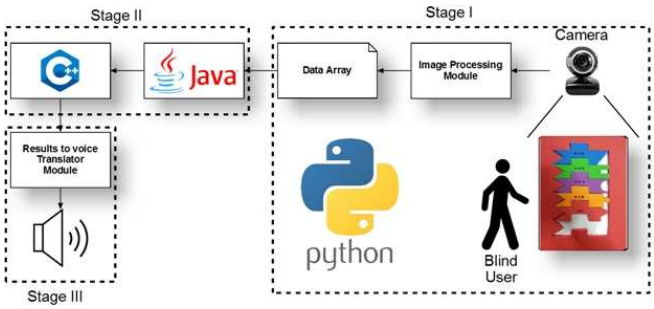


Figure 2: Program processing stages.
Source: Prepared by the Author

Stage I is developed in Python for process the frames recovered from the physical programming space where the user interacts, uses image-processing techniques to generate a data array containing blocks information (color, sequence, and gaps).

Stage II uses Java and C++ to implement the converter module (translator) and generate the source code of each program, which will then be run using a standard compiler.

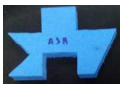
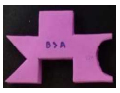
The final Stage III is developed in Python also, complements the other two phases by converting the compilation results into voice signals that report the results of program execution back to the user.

C. *Program Blocks*

Blocks representing program functions or statements are characterized by different colors, textures, and shapes, with the latter two being discriminators through which the child identifies each statement.

The following are examples of six program blocks and the function they represent:

TABLE 1: Program blocks and the functions they represent

Block	Program Function
	Represents a comparison function: If A > B
	Represents a comparison function: If B > A
	Represents division by 2 with decimals: N/2
	Represents the multiplication of two numbers: A * B
	Represents the sum of two numbers: A + B
	Represents the screen output block: PRINT (Result)

Source: Prepared by the Author

The blocks are inserted in a larger container as a puzzle, depending on the program structure the user wants to represent.

III. PROCESSING TECHNIQUES

To properly generate the program code, the position and color of each block must be determined.

A. *Determining Block Positions*

Block positions are determined by a vertical scan of the programming canvas.

The blocks are scanned throught the camera to determine their position within a program. In addition, this scan will also determine the program opening and closing blocks. Hence, a program that does not include a result output block will be immediately rejected and not compiled (as it will not generate results).

B. *Determining Block Colors*

To determine block colors, the “color space change” technique was used. As it only employs three parameters, the RGB format was not used because filters become highly susceptible to error.

The HSV color space is a model used to represent color spaces based on three components: H (Hue), the color type or tint; S (Saturation), which can be understood as the shade of color we see, and V (Value), which describes the color brightness or intensity.

The main advantage of using the HSV model over RGB is that color filters may be established using only H values, which represent the color type.

Because the HSV model uses polar coordinates, a linear transformation to the RGB format must be used. To convert a pixel’s RGB values into HSV, the following relationships were used:

Authorized licensed use limited to: University of New South Wales. Downloaded on December 11,2025 at 01:41:18 UTC from IEEE Xplore. Restrictions apply.

$v = max$

$$s = \begin{cases} 0, & if\ \max = 0 \\ \dfrac{\max - min}{max} = 1 - \dfrac{min}{max}, & otherwise \end{cases}$$

$$h = \begin{cases} 0 & if\ \max = min \\ \left(60° \ x\ \dfrac{g-b}{max-min} + 0°\right) mod\ 360°, & if\ \max = r \\ \left(60° \ x\ \dfrac{b-r}{max-min} + 120°\right), & if\ \max = g \\ \left(60° \ x\ \dfrac{r-g}{max-min} + 240°\right), & if\ \max = b \end{cases}$$

Where ***max = max(R,G,B)*** *and* ***min = min(R,G,B)***

The algorithm used for converting RGB into HSV, (BasketBall Robot, 2005) is presented below.

```
//r,g,b values are from 0 to 1
//h= [0,360], s=[0,1], v=[0,1]
//if s=0 then h=-1 (undefined)
void RGBtoHSV(float r, float g, float b, float *h, float *s, float
*v){
        float min, max, delta;
        min = MIN(r,g,b);
        max = MAX(r,g,b);
        *v=max;                 //v
        delta = max- min;
        if(max!=0){
                *s=delta/max; //s
        }else{
                //r=g=b=0   //s=0, v is undefined
                *s=0;
                *h=-1;
                return;
        }
        if(r==max){
          *h=(g-b)/delta;   //between yellow & magenta
        }else if(g==max){
          *h=2+(b-r)/delta; //between cyan & yellow
        }else{
          *h=4+(r-g)/delta; //between magenta & cyan
        }
        *h=*=60;            //degrees
          if(*h<0){
            *h+=360;}
    }
```

C. Variable Initialization

This prototype version works with two variables, A and B, which can be initialized with several integer values, as required, through two potentiometers and an Arduino Mega interface.

As the values of A and B increase, a variable-frequency buzzer generates a sound proportional to the value of the variable. To differentiate A from B, each variable uses a different base frequency.

The original values of potenciometer (0-1023) is scaled from 0 to 100 for feasible values of variables.

D. Source Code Generation

Such as is showed in Figure 2, source Code Generation represents Stage II of the process. The translator module receives a data array containing the blocks identified by the camera as input and the initial values for variables A and B entered through the Arduino interface.

The translator module then generates a standard C++ source code. The generated program can be verified and compiled in the same way as any other program that is entered manually in the development environment (e.g. DevC++, CodeBlocks)

IV. Experimentation

Next, we show some programs programs that can be generated using the proposed prototype, however any kind of program can be generated modifying the block's position and the number of blocks used.

a) Example 1: calculate the arithmetic mean of two numbers
b) Example 2: calculate the area of a triangle.
c) Example 3: if A > B, then add, else if B > A, then multiply.
d) Example 4: If B > A, then multiply and divide by 2.
e) Example 5: if B > A, then add. Else, if A > B, then do nothing.

The following figures shows the implementation of b) and d) examples



Figure 3: Example 1: calculate the area of a triangle.
Source: Prepared by the Author

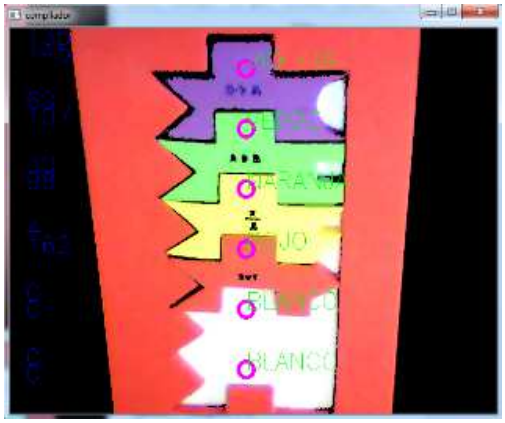


Figure 4:Example 2: If B > A,then multiply and divide by 2.
Source: Prepared by the Author

In the previous example, a blank box represents the “then do nothing” instruction requested in the algorithm.

Authorized licensed use limited to: University of New South Wales. Downloaded on December 11,2025 at 01:41:18 UTC from IEEE Xplore. Restrictions apply.

V. Conclusions

- Through image processing, a translator may be implemented for the generation of source code in a standard programming language such as C++ from a program built using physical blocks. This tool facilitates coding for the visually impaired childrens, who are not able access a computer by conventional means (e.g., a keyboard), thus allowing them to effectively develop their computational thinking skills.
- When determining block colors, the technique of changing the color space from RGB to HSV considerably simplifies the detection process because color filters can be implemented using only one component, which is the H parameter.
- In this study, the first prototype used six blocks of different colors, with each one representing a different program function. However, in the future, this prototype could be extended to use additional blocks with more colors. In fact, the prototype could even support using blocks of the same color but of different shades, as they could be differentiated using the H and S filter parameters.
- On the other hand, variable initialization implies a wide range of possibilities. Accordingly, as their values increase, variable values are identified through a buzzer that vibrates at a frequency that is proportional to each variable value. This prototype uses only two variables: A and B. However, the system can support as many variables as the existing number of analog inputs provided by the Arduino Mega interface (16 inputs).
- In addition, since image-based processing is being used, input quality is extremely important. Therefore, we recommend using a camera with a resolution (at least 8 MP). Likewise, environmental lighting and contrast conditions, as well as the canvas position with respect to the camera, are parameters that must be calibrated and kept constant at the place where the prototype is operating.
- Because the resulting code is generated in a standard language, such as C++, the child's computational thinking skills may be assessed as strictly as they would be assessed for any other programmer. In fact, this code may even be transparently shared and reused by other programmers.
- Finally, since the prototype reads the program created out loud, instruction by instruction, as well as the final results, the child receives immediate feedback on their solution proposed. Moreover, in the case of errors, the child can use the instructions that are read to debug the program.

References

[1] Almonacid, V., & Carrasco, M. J. (1989). *El juego en los niños ciegos y deficientes visuales*. [Using games for blind and visually impaired children] http://sid.usal.es/idocs/F8/FDO12402/juego_ni%F1os_ciegos_deficientes_visuales.pdf

[2] Costa, R., Araújo, C., & Henriques, P. R. (2021). Melodic - Teaching computational thinking to visually impaired kids. OpenAccess Series in Informatics. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl Publishing, Germany. http://doi.org/10.4230/OASIcs.ICPEC.2021.4

[3] Milne, L. R., & Ladner, R. E. (2018). Blocks4All: Overcoming accessibility barriers to blocks programming for children with visual impairments. Conference on Human Factors in Computing Systems - Proceedings, 2018–April. http://doi.org/10.1145/3173574.3173643

[4] Basketball Robot. (2005). Converting from RGB to HSV. Mechatronics Lab final project. http://coecsl.ece.illinois.edu/ge423/spring05/group8/finalproject/hsv_writeup.pdf

[5] Basogain Olabe, X., Olabe Basogain, M. A., & Olabe Basogain, J. C. (2015). Pensamiento computacional a través de la programación: paradigma de aprendizaje [Computational Thinking through Programming: A Learning Paradigm]. RED-Revista de Educación a Distancia 46(6). https://www.um.es/ead/red/46/Basogain.pdf

[6] Bora, D. J., Gupta, A. K., & Khan, F. A. (2015). Comparing the performance of L*A*B* and HSV color spaces with respect to color image segmentation, *5*(2), 192–203. http://arxiv.org/abs/1506.01472

[7] Chen, Y., Xie, Y., & Wang, Y. (2013). Detection and recognition of traffic signs based on HSV vision model and shape features. *Journal of Computers, 8*(5), 1366–1370. http://www.jcomputers.us/vol8/jcp0805-35.pdf

[8] Elia, D. E. (2014). Niños creadores de tecnología [Children as Creators of Technology]. *Actas Del Congreso Iberoamericano de Ciencia, Tecnología Y Educación* (1–14). https://www.oei.es/historico/congreso2014/memoriactei/469.pdf

[9] Ganesan, P., Rajini, V., Sathish, B. S., & Shaik, K. B. (2014). HSV color space-based segmentation of region of interest in satellite images. *International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT), Kanyakumari, 2014,* 101–105. http://ieeexplore.ieee.org.ezproxy.ulima.edu.pe/stamp/stamp.jsp?tp=&arnumber=6992938&isnumber=6992918

[10] Gurises Unidos. (2011). Pensamiento computacional: un aporte para la educación de hoy [Computational Thinking: A Contribution to Current Education]. *Fundación Telefónica*, 1–116. https://www.gurisesunidos.org.uy/wp-content/uploads/2017/11/PensamientoComputacional.pdf

[11] Hernández, N. (2020). *Cómo aprenden a programar los niños con déficit visual* [How Visually Impaired Children Learn to Program]. Educación 3.0. https://www.educaciontrespuntocero.com/noticias/como-aprenden-programar-ninos-deficit-visual/

[12] Herranz Tardón, A., Holgado López, M. D., & Marín Palomino, C. (2013). «¡Voy al cole!» Material para la inclusión del niño con discapacidad visual en la etapa de educación infantil *Integración. Revista Sobre Discapacidad Visual*, *62*, 1–17. https://www.once.es/dejanos-ayudarte/la-discapacidad-visual/revista-integracion/2013-integracion-62/numero-62/62-03-herranz-holgado-marin-voy-al-cole-material.pdf/download

[13] Kolkur, S., Kalbande, D., Shimpi, P., Bapat, C., & Jatakia, J. (2017). *Human skin detection using RGB, HSV and YCbCr color models*, *137*, 324–332. https://doi.org/10.2991/iccasp-16.2017.51

[14] Linn, A. (2020). With Code Jumper, experts look to jumpstart computer science interest for kids who are blind. Microsoft. https://news.microsoft.com/innovation-stories/project-torino-code-jumper/

[15] Nieves. (2014). Diseño de imágenes para ciegos, material didáctico para niños con discapacidad visual [Designing of Images for the Blind. Academic Materials for Visually Impaired Children] 1–412. https://riunet.upv.es/bitstream/handle/10251/37882/FUENTES%20-%20Dise%c3%b1o%20de%20im%c3%a1genes%20para%20ciegos%2c%20material%20did%c3%a1ctico%20para%20ni%c3%b1os%20con%20discapacidad%20visual.pdf?sequence=1&isAllowed=y

[16] Raya, A. (2020). El lenguaje de programación "físico" que permite que niños ciegos creen sus propios programas. [The "physical" programming language that allows blind children to create their own programs] *El Español*. https://www.elespanol.com/omicrono/tecnologia/20190123/lenguaje-programacion-fisico-permite-ciegos-propios-programas/370714482_0.html

[17] Riaz, M., Kang, G., Kim, Y., Pan, S., & Park, J. (2008). Efficient image retrieval using adaptive segmentation of HSV color space. International Conference on Computational Sciences and Its Applications, Perugia, 2008, 491–496. http://ieeexplore.ieee.org.ezproxy.ulima.edu.pe/stamp/stamp.jsp?tp=&arnumber=4561254&isnumber=4561187

Authorized licensed use limited to: University of New South Wales. Downloaded on December 11,2025 at 01:41:18 UTC from IEEE Xplore. Restrictions apply.